

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 3

Выполнила:

Космач Мария

Группа К3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

- реализовать автодокументирование средствами swagger;
- реализовать документацию API средствами Postman.

Ход работы

Так как для написания лабораторной работы 2 использовался фреймворк tsoa, то автодокументирование swagger происходило путём добавления аннотаций к контроллеру и его методам. На рисунке 1 представлен AdvertisementController, где можно посмотреть пример аннотаций, а на рисунках 2, 3, 4 – сам swagger.

На рисунке 5 представлена коллекция Postman

```

@Route("properties") Show usages
@Tags("Properties")
export class PropertyController extends Controller {
    @Get() Show usages
    @SuccessResponse("200", "Ok")
    public async getAllProperties(): Promise<PropertyEntity[]> {
        return propertyService.getAll();
    }

    @Get("{id}") Show usages
    @SuccessResponse("200", "Ok")
    @Response<EntityNotFoundErrorResponse>(404, "Property not found")
    public async getPropertyById(@Path() id: number): Promise<PropertyEntity> {
        return propertyService.getById(id);
    }

    @Post() Show usages
    @SuccessResponse("201", "Created")
    @Security("jwt")
    public async create(@Body() body: CreatePropertyRequest, @Request() req: any): Promise<PropertyEntity> {
        return propertyService.create(body, req.user.id);
    }

    @Put("{id}") Show usages
    @SuccessResponse("200", "Updated")
    @Response<EntityNotFoundErrorResponse>(404, "Property not found")
    public async update(
        @Path() id: number,
        @Body() body: UpdatePropertyRequest
    ): Promise<PropertyEntity> {
        return propertyService.update(id, body);
    }

    @Delete("{id}") Show usages
    @SuccessResponse("204", "Deleted")
    @Response<EntityNotFoundErrorResponse>(404, "Property not found")
    public async delete(@Path() id: number): Promise<void> {
        await propertyService.delete(id);
        return;
    }
}

```

Рисунок 1 – AdvertisementController

GET

/properties/{id}

⌵

Parameters

Cancel

Name	Description
id * required	
number(\$double)	2
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:3000/properties/2' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:3000/properties/2
```

Server response

Code	Details
Undocumented	TypeError: Load failed

Responses

Code	Description	Links
200	Ok	No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
{
  "id": 0,
  "owner": {
    "id": 0,
    "mail": "string",
    "password": "string",
    "firstName": "string",
    "lastName": "string",
    "createdAt": "2025-09-14T14:37:25.423Z",
    "updatedAt": "2025-09-14T14:37:25.423Z"
  },
  "title": "string",
  "description": "string",
  "rentType": "day",
  "status": "active",
  "price": 0,
  "photos": [
    {
      "id": 0,
      "property": "string",
      "path": "string"
    }
  ],
  "messages": [
```

Рисунок 2 – Метод для получения объявления в swagger

Servers

/ ▾

Authorize



User ^

GET	/users	🔒 ▾
GET	/users/me	🔒 ▾
GET	/users/mail	🔒 ▾
GET	/users/{userId}	🔒 ▾
PUT	/users/{userId}	🔒 ▾
DELETE	/users/{userId}	🔒 ▾

Reviews ^

GET	/reviews/{id}	▾
DELETE	/reviews/{id}	▾
GET	/reviews/rental-agreement/{rentalAgreementId}	▾
POST	/reviews	▾

RentalAgreements ^

GET	/rental-agreements/{id}	▾
DELETE	/rental-agreements/{id}	▾
GET	/rental-agreements/renter/{renterId}	▾
POST	/rental-agreements	▾
PUT	/rental-agreements/{id}/status	▾

Рисунок 3 – Swagger

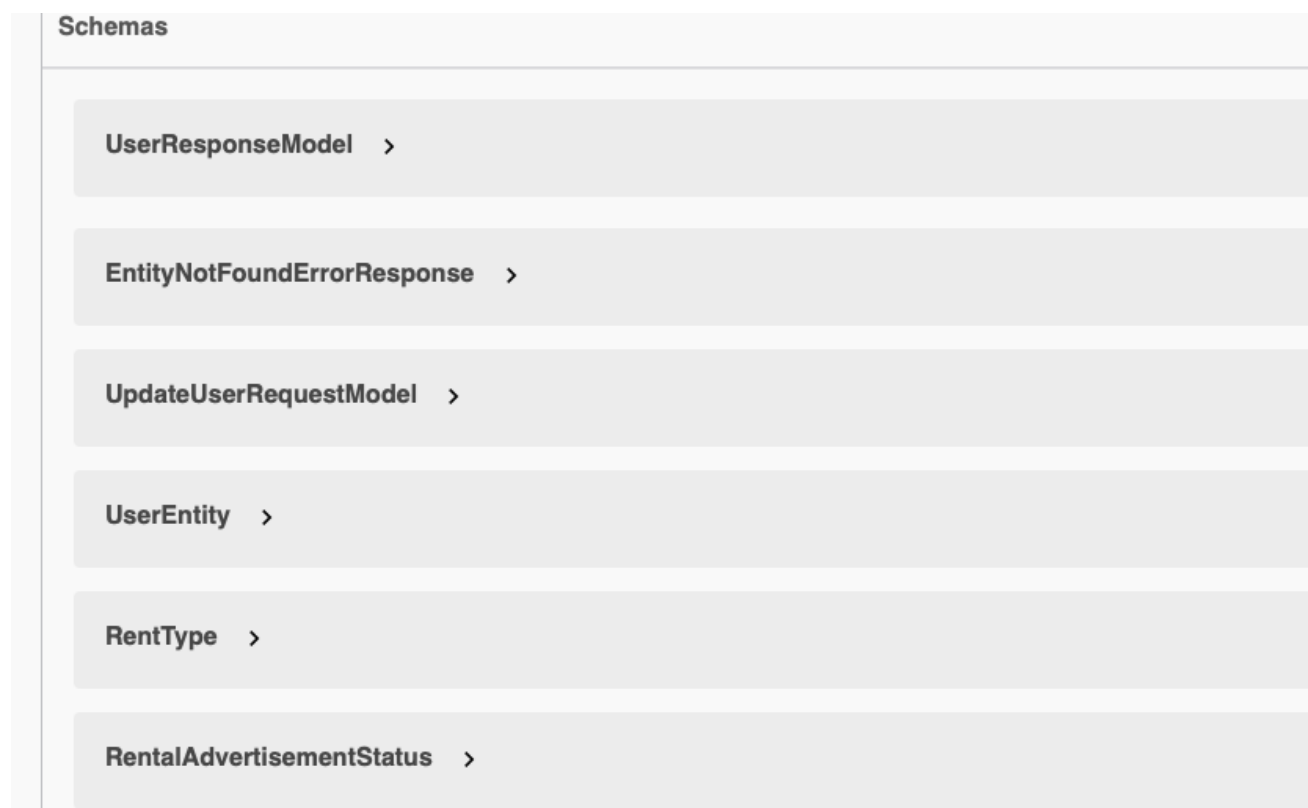


Рисунок 4 – Примеры моделей в Swagger

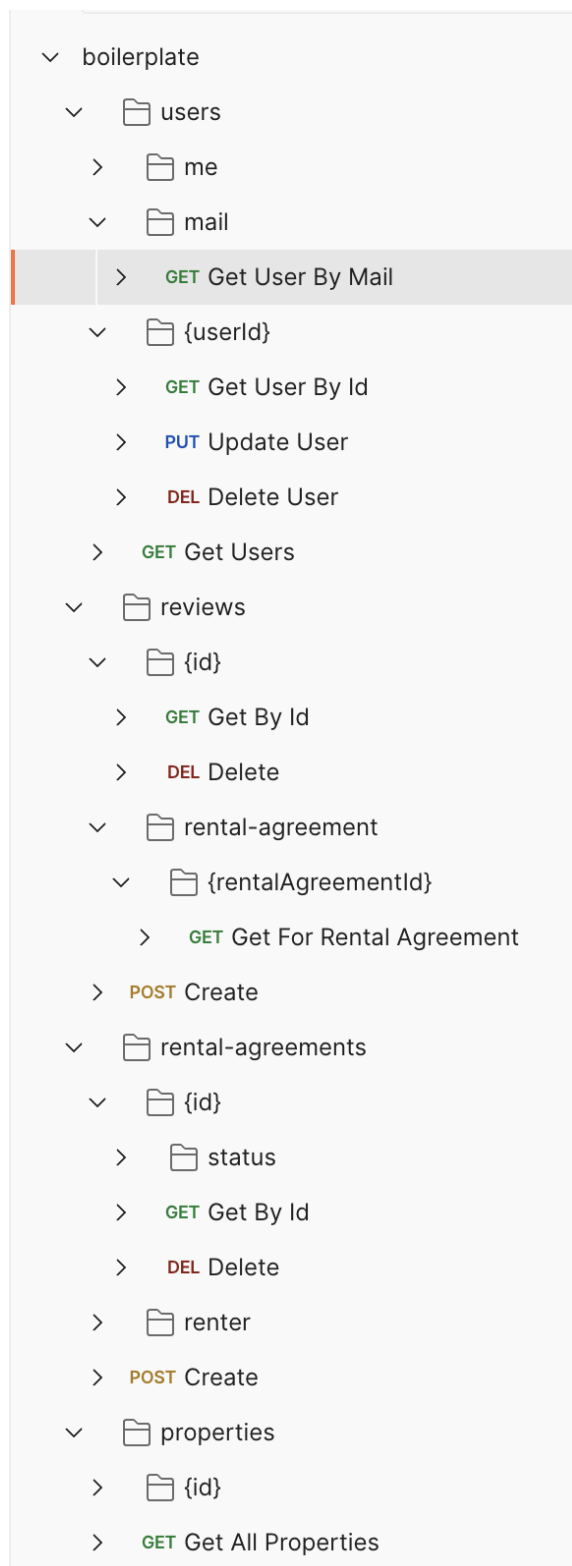


Рисунок 5 – Коллекция в PostMan

Вывод

В рамках работы реализована автодокументация swagger при помощи фреймворка tsoa, а также создана коллекция запросов в postman.